

Task 2: Employee Attrition and Performance Prediction (Classification)

PREPROCESSING

```
In [84]: #Loading the dataset
import pandas as pd

df=pd.read_csv("../Data/WA_Fn-UseC_-HR-Employee-Attrition.csv")
df.head()
```

```
Out[84]:
```

	Age	Attrition	BusinessTravel	DailyRate	Department	DistanceFromHome	Educational
0	41	Yes	Travel_Rarely	1102	Sales		1
1	49	No	Travel_Frequently	279	Research & Development		8
2	37	Yes	Travel_Rarely	1373	Research & Development		2
3	33	No	Travel_Frequently	1392	Research & Development		3
4	27	No	Travel_Rarely	591	Research & Development		2

5 rows × 35 columns



```
In [85]: df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1470 entries, 0 to 1469
Data columns (total 35 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Age                                   1470 non-null   int64
1   Attrition                           1470 non-null   object
2   BusinessTravel                      1470 non-null   object
3   DailyRate                           1470 non-null   int64
4   Department                          1470 non-null   object
5   DistanceFromHome                   1470 non-null   int64
6   Education                           1470 non-null   int64
7   EducationField                      1470 non-null   object
8   EmployeeCount                      1470 non-null   int64
9   EmployeeNumber                     1470 non-null   int64
10  EnvironmentSatisfaction             1470 non-null   int64
11  Gender                             1470 non-null   object
12  HourlyRate                          1470 non-null   int64
13  JobInvolvement                     1470 non-null   int64
14  JobLevel                           1470 non-null   int64
15  JobRole                             1470 non-null   object
16  JobSatisfaction                     1470 non-null   int64
17  MaritalStatus                      1470 non-null   object
18  MonthlyIncome                      1470 non-null   int64
19  MonthlyRate                         1470 non-null   int64
20  NumCompaniesWorked                 1470 non-null   int64
21  Over18                             1470 non-null   object
22  OverTime                           1470 non-null   object
23  PercentSalaryHike                  1470 non-null   int64
24  PerformanceRating                  1470 non-null   int64
25  RelationshipSatisfaction            1470 non-null   int64
26  StandardHours                      1470 non-null   int64
27  StockOptionLevel                   1470 non-null   int64
28  TotalWorkingYears                  1470 non-null   int64
29  TrainingTimesLastYear              1470 non-null   int64
30  WorkLifeBalance                    1470 non-null   int64
31  YearsAtCompany                     1470 non-null   int64
32  YearsInCurrentRole                 1470 non-null   int64
33  YearsSinceLastPromotion             1470 non-null   int64
34  YearsWithCurrManager                1470 non-null   int64
dtypes: int64(26), object(9)
memory usage: 402.1+ KB

```

```
In [86]: df.describe()
```

Out[86]:

	Age	DailyRate	DistanceFromHome	Education	EmployeeCount	Empl
--	-----	-----------	------------------	-----------	---------------	------

count	1470.000000	1470.000000	1470.000000	1470.000000	1470.0	
mean	36.923810	802.485714	9.192517	2.912925	1.0	
std	9.135373	403.509100	8.106864	1.024165	0.0	
min	18.000000	102.000000	1.000000	1.000000	1.0	
25%	30.000000	465.000000	2.000000	2.000000	1.0	
50%	36.000000	802.000000	7.000000	3.000000	1.0	
75%	43.000000	1157.000000	14.000000	4.000000	1.0	
max	60.000000	1499.000000	29.000000	5.000000	1.0	

8 rows × 26 columns



Data Cleaning

```
In [87]: # check for missing data
print(df.isnull().sum())
```

Age	0
Attrition	0
BusinessTravel	0
DailyRate	0
Department	0
DistanceFromHome	0
Education	0
EducationField	0
EmployeeCount	0
EmployeeNumber	0
EnvironmentSatisfaction	0
Gender	0
HourlyRate	0
JobInvolvement	0
JobLevel	0
JobRole	0
JobSatisfaction	0
MaritalStatus	0
MonthlyIncome	0
MonthlyRate	0
NumCompaniesWorked	0
Over18	0
OverTime	0
PercentSalaryHike	0
PerformanceRating	0
RelationshipSatisfaction	0
StandardHours	0
StockOptionLevel	0
TotalWorkingYears	0
TrainingTimesLastYear	0
WorkLifeBalance	0
YearsAtCompany	0
YearsInCurrentRole	0
YearsSinceLastPromotion	0
YearsWithCurrManager	0

dtype: int64

```
In [88]: # checking for negative value
num_df=df.select_dtypes(include='number')
negative_value=num_df < 0
negative_count=negative_value.sum()
print(negative_count)
```

```

Age 0
DailyRate 0
DistanceFromHome 0
Education 0
EmployeeCount 0
EmployeeNumber 0
EnvironmentSatisfaction 0
HourlyRate 0
JobInvolvement 0
JobLevel 0
JobSatisfaction 0
MonthlyIncome 0
MonthlyRate 0
NumCompaniesWorked 0
PercentSalaryHike 0
PerformanceRating 0
RelationshipSatisfaction 0
StandardHours 0
StockOptionLevel 0
TotalWorkingYears 0
TrainingTimesLastYear 0
WorkLifeBalance 0
YearsAtCompany 0
YearsInCurrentRole 0
YearsSinceLastPromotion 0
YearsWithCurrManager 0
dtype: int64

```

```

In [89]: # checking for unrealistic age experience
entry_age=df['Age'] - df['TotalWorkingYears']
condition=entry_age<14 # if total age minus working years is lower than 14 that
unrealistic_employee=df[condition] #true where exp is unrealistic and picks up t
print(unrealistic_employee)

```

Empty DataFrame

Columns: [Age, Attrition, BusinessTravel, DailyRate, Department, DistanceFromHome, Education, EducationField, EmployeeCount, EmployeeNumber, EnvironmentSatisfaction, Gender, HourlyRate, JobInvolvement, JobLevel, JobRole, JobSatisfaction, MaritalStatus, MonthlyIncome, MonthlyRate, NumCompaniesWorked, Over18, OverTime, PercentSalaryHike, PerformanceRating, RelationshipSatisfaction, StandardHours, StockOptionLevel, TotalWorkingYears, TrainingTimesLastYear, WorkLifeBalance, YearsAtCompany, YearsInCurrentRole, YearsSinceLastPromotion, YearsWithCurrManager]

Index: []

[0 rows x 35 columns]

CATEGORICAL & ORDINAL REPRESENTATION

```

In [90]: nominal_cols = ['Department', 'JobRole', 'BusinessTravel', 'Gender', 'MaritalStatus']
ordinal_cols = ['Education', 'JobLevel', 'JobSatisfaction', 'EnvironmentSatisfaction', 'WorkLifeBalance', 'RelationshipSatisfaction', 'StockOptionLevel']

from sklearn.preprocessing import OneHotEncoder, OrdinalEncoder

nominal_encoded=OneHotEncoder(sparse_output=False).fit_transform(df[nominal_cols])
ordinal_encoded=OrdinalEncoder().fit_transform(df[ordinal_cols])

print(nominal_encoded.shape)
print(ordinal_encoded.shape)

```

(1470, 22)
(1470, 8)

In [91]: `df.describe()`

Out[91]:

	Age	DailyRate	DistanceFromHome	Education	EmployeeCount	Em
count	1470.000000	1470.000000	1470.000000	1470.000000	1470.0	
mean	36.923810	802.485714	9.192517	2.912925	1.0	
std	9.135373	403.509100	8.106864	1.024165	0.0	
min	18.000000	102.000000	1.000000	1.000000	1.0	
25%	30.000000	465.000000	2.000000	2.000000	1.0	
50%	36.000000	802.000000	7.000000	3.000000	1.0	
75%	43.000000	1157.000000	14.000000	4.000000	1.0	
max	60.000000	1499.000000	29.000000	5.000000	1.0	

8 rows × 26 columns



FEATURE SCALING

In [92]:

```
scaling_cols = ['Age', 'MonthlyIncome', 'MonthlyRate', 'DailyRate',  
               'HourlyRate', 'DistanceFromHome', 'NumCompaniesWorked',  
               'PercentSalaryHike', 'TotalWorkingYears', 'TrainingTimesLastY',  
               'YearsAtCompany', 'YearsInCurrentRole', 'YearsSinceLastPromot',  
               'YearsWithCurrManager']  
  
from sklearn.preprocessing import RobustScaler #Robust scaler works best with ou  
  
scaler_encoding=RobustScaler().fit_transform(df[scaling_cols])  
print(scaler_encoding.shape)
```

(1470, 14)

FEATURE SELECTION

In [93]:

```
corr_matrix = df.corr(numeric_only=True).abs()  
high_corr = corr_matrix[corr_matrix > 0.85]  
print(high_corr.stack().reset_index())
```

	level_0	level_1	0
0	Age	Age	1.0000
1	DailyRate	DailyRate	1.0000
2	DistanceFromHome	DistanceFromHome	1.0000
3	Education	Education	1.0000
4	EmployeeNumber	EmployeeNumber	1.0000
5	EnvironmentSatisfaction	EnvironmentSatisfaction	1.0000
6	HourlyRate	HourlyRate	1.0000
7	JobInvolvement	JobInvolvement	1.0000
8	JobLevel	JobLevel	1.0000
9	JobLevel	MonthlyIncome	0.9503
10	JobSatisfaction	JobSatisfaction	1.0000
11	MonthlyIncome	JobLevel	0.9503
12	MonthlyIncome	MonthlyIncome	1.0000
13	MonthlyRate	MonthlyRate	1.0000
14	NumCompaniesWorked	NumCompaniesWorked	1.0000
15	PercentSalaryHike	PercentSalaryHike	1.0000
16	PerformanceRating	PerformanceRating	1.0000
17	RelationshipSatisfaction	RelationshipSatisfaction	1.0000
18	StockOptionLevel	StockOptionLevel	1.0000
19	TotalWorkingYears	TotalWorkingYears	1.0000
20	TrainingTimesLastYear	TrainingTimesLastYear	1.0000
21	WorkLifeBalance	WorkLifeBalance	1.0000
22	YearsAtCompany	YearsAtCompany	1.0000
23	YearsInCurrentRole	YearsInCurrentRole	1.0000
24	YearsSinceLastPromotion	YearsSinceLastPromotion	1.0000
25	YearsWithCurrManager	YearsWithCurrManager	1.0000

```
In [94]: # JobLevel and MonthlyIncome - 0.95 - extremely high, we drop one collinear feat
df.drop(columns=['JobLevel'], inplace=True)
```